

- for any Web application.
2. It supports different Web drivers for various browsers. These drivers are the Firefox Web driver for Mozilla Firefox, Chrome's Web driver for Google Chrome, PhantomJs Web driver for PhantomJs, zope.testbrowser for ZopeTest and a remote Web driver for different 'headless' (with no GUI) testing.
3. Splinter also allows us to find different elements in any Web page by their XPath, CSS, tag value, name, ID, text or value. In case we need more accurate control of the Web page or we need to do something more, such as interacting with old «frameset» tags, Splinter even exposes the Web driver that allows us to use the low level methods used for interacting with that tag.
4. Splinter supports multiple Web automation back-ends. We can use the same set of test code for doing browser-based testing with Selenium as its back-end, and for 'headless' testing with zope.testbrowser as its back-end.
5. It has extensive support for using iframes and interacts with them by just passing the iframe's name, ID or index value. There is also Chrome support for various alerts and prompts in the Splinter 0.4 version.
6. We can easily execute JavaScript in different drivers which support Splinter. We can even return the result of the script using an inbuilt method called *evaluate_script*.
7. Splinter has got the ability to work with AJAX and asynchronous JavaScript using various inbuilt methods.
8. When we use Splinter to work with AJAX and asynchronous JavaScript, it's a common experience to have some elements which are not present in HTML code (since they are created using JavaScript, dynamically). In such cases, we can use various inbuilt methods such as *is_element_present* or *is_text_present* for checking the existence of any specific element or text. Splinter will actually load the HTML and the JavaScript in the browser, and the check will be performed before JavaScript is processed.
9. The Splinter project has full documentation for its APIs and this is really important when we have to deal with different third party libraries.
10. We can also easily set up a Splinter development environment. We need to make sure we have some basic development tools in our machine, before setting up an entire environment with just one command.
11. There is also a provision for creating a new Splinter browser in an easy and simple way. We just need to implement a test case for this.
12. Using Splinter, it's possible to check the HTTP status code that a browser visits. We can use the *status_code*, *is_success* method to do the work for us. We can compare the status code directly.
13. Whenever we use the *visit* method, Splinter actually checks if the given response is a *success* or *not*, and if it is not, then Splinter raises an *HttpResponseError* exception.

- This helps to confirm if the given response is okay or not.
14. It is possible to manipulate the cookies that are using the cookies' attributes from any browser instance. The cookie's attribute is actually an instance of a *CookieManager* class which manipulates cookies, such as adding and deleting them.
15. One can create new drivers using Splinter. For instance, if we need to create a new Splinter browser, we just need to implement a test case (extending *test.base.BaseBrowserTests*). All this will be present in a Python file, which will act as a driver for any future usage.

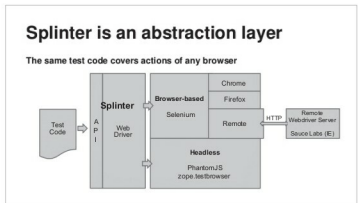


Figure 1: Flow diagram for Splinter acting as an abstraction layer
(Image source: googleimages.com)

Drivers supported by Splinter

Drivers play a significant role when it comes to any Web application. In Splinter, a Web driver helps us open that specific application whose driver we are using. Different types of drivers are supported by Splinter, based on the way any specific application is accessed and tested. There are browser based drivers, which help to open specific browsers; apart from that we have headless drivers, which help in headless testing and then there are remote drivers, which help to connect to any Web application present on a remote machine. Here is a list of drivers that are supported by Splinter.

Browser based drivers:

- Chrome WebDriver
- Firefox WebDriver
- Remote WebDriver

Headless drivers

- Chrome WebDriver
- Phantomjs WebDriver
- zope.testbrowser
- Django client
- Flask client

Remote driver

- Remote WebDriver

Prerequisites and installation of Splinter

To install Splinter, Python 2.7 or above should be installed on the system. We can download Python from <http://www.python.org>. Make sure you have already set up your development environment.